

AI Smart Traffic Monitoring and Emergency Priority System

Project report submitted to

Shri Ramdeobaba College of Engineering & Management, Nagpur

in partial fulfillment of requirement for the award of degree of

Bachelor of Technology (B.Tech)

in

COMPUTER SCIENCE AND ENGINEERING

(DATA SCIENCE)

by

Pranav Tapdiya (Roll no. 08)

Pushkar Sawarkar Thakur (Roll no. 13)

Pushpak Bajanghate (Roll no. 14)

Samiksha Shete (Roll no. 28)

Ashlesha Varghane (Roll no. 69)

of

VI Semester

Under the Guidance of

Prof. R. M. Rewatkar

Department of Data Science

Shri Ramdeobaba College of Engineering & Management, Nagpur 440 013

*(An Autonomous Institute affiliated to Rashtrasant Tukdoji Maharaj Nagpur University,
Nagpur)*

May 2026

**SHRI RAMDEOBABA COLLEGE OF ENGINEERING AND
MANAGEMENT, NAGPUR**

(An Autonomous Institute affiliated to Rashtrasant Tukdoji Maharaj Nagpur University, Nagpur)

Department of Data Science

CERTIFICATE

This is to certify that the project report on “*AI Smart Traffic Monitoring and Emergency Priority System*” is a Bonafide work of **Pranav Khandelwal, Sudhanshu Thakur, Sujal Sakla, Manas Doye and Ayush Lambat** submitted to Rashtrasant Tukdoji Maharaj Nagpur University, Nagpur in partial fulfilment of the award of Degree of Bachelor of Technology (B.Tech), in Computer Science and Engineering (Data Science). It has been carried out in Department of Data Science at Shri Ramdeobaba College of Engineering and Management, Nagpur during the academic year 2025-2026.

Date:

Place: Nagpur

Prof. R. M. Rewatkar Project Guide Department of Data Science	Dr. A. M. Karandikar H.o.D Data Science
---	--

Dr. M. B. Chandak

Principal

DECLARATION

We hereby declare that the project report titled “**AI Smart Traffic Monitoring and Emergency Priority System**” submitted herein, has been carried out in the Department of Data Science, Shri Ramdeobaba College of Engineering and Management, Nagpur. The work is original and has not been submitted earlier as a whole or part for the award of any degree/diploma at this or any other institution / University.

Date:

Place: Nagpur

Pranav Tapdiya (Roll no. 08)

Pushkar Sawarkar Thakur (Roll no. 13)

Pushpak Bajanghate (Roll no. 14)

Samiksha Shete(Roll no. 28)

Ashlesha Varghane (Roll no. 69)

APPROVAL SHEET

This report entitled **AI Smart Traffic Monitoring and Emergency Priority System** by **Pranav Khandelwal, Sudhanshu Thakur, Sujal Sakla, Manas Doye and Ayush Lambat** is approved for the degree of Bachelor of Technology (B.Tech) in Computer Science and Engineering (Data Science).

Prof. R. M. Rewatkar Project Guide	External Examiner
---	-------------------------

Dr. A. M. Karandikar

H.o.D

Data Science

Date: _____

Place: Nagpur

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to our project guide, Prof. R. M. Rewatkar for their invaluable guidance, continuous support, and encouragement throughout the development of this project. Their insights and constructive feedback have been instrumental in shaping the direction and quality of our work.

We are also grateful to Dr. A. M. Karandikar, Head of Department of Data Science, and the entire faculty of the Department of Data Science at Shri Ramdeobaba College of Engineering and Management, Nagpur, for providing us with the resources and infrastructure required for this project.

We extend our thanks to Rashtrasant Tukdoji Maharaj Nagpur University for their academic framework and to the open-source community whose tools and libraries formed the backbone of this system — including FastAPI, YOLOv8, OpenCV, React, and Ultralytics.

Finally, we thank our families and friends for their patience, motivation, and unwavering support throughout this journey.

Date: _____

Place: Nagpur

ABSTRACT

The AI Smart Traffic Monitoring and Emergency Priority System is a production-ready, real-time traffic intelligence platform designed for urban intersections, with a default deployment profile targeting Variety Square, Nagpur. The system integrates computer vision-based vehicle detection, live congestion estimation, adaptive signal control, and emergency-priority routing into a unified full-stack application accessible via a modern web dashboard.

The core of the system is a YOLOv8-based detection and multi-object tracking pipeline that processes live video frames from webcam, RTSP, or video file sources. Vehicle classes detected include cars, bikes, buses, trucks, ambulances, and fire brigade vehicles. Lane-level and intersection-level traffic loads are computed in real time and classified into LOW, MEDIUM, or HIGH congestion states. Emergency vehicle detection automatically triggers green-corridor logic, enabling priority routing across the monitored intersection network.

The backend is built using FastAPI and Uvicorn, exposing REST endpoints and WebSocket streams. A React 18 frontend, powered by Vite and Tailwind CSS, renders a live multi-panel dashboard featuring an interactive map (Google Maps and Leaflet), camera feed, alert timeline, solar power telemetry, and environmental monitoring panels. All panels receive real-time updates via WebSocket with REST polling as fallback.

The system is designed for Windows-first local development with ready-to-run launch scripts, and includes deployment configuration for cloud platforms including Railway, Render, Netlify, and Vercel.

Keywords: Computer Vision, YOLOv8, Object Tracking, Congestion Detection, Emergency Vehicle Priority, FastAPI, WebSocket, React, Smart City, Traffic Signal Control

TABLE OF CONTENTS

Content	Page No.
Declaration	ii
Approval Sheet	iii
Acknowledgements	iv
Abstract	v
Table of Contents	vi
List of Figures	viii
List of Tables	viii
Chapter 1: Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Objectives	2
1.4 Scope of the Project	2
1.5 Organization of Report	2
Chapter 2: Literature Review	3
Chapter 3: System Analysis and Design	5
3.1 System Architecture	5
3.2 Technology Stack	6
3.3 Data Flow Diagram	6
3.4 Use Case Diagram	7
Chapter 4: System Implementation	9
4.1 Backend Implementation (FastAPI)	9
4.2 YOLOv8 Detection and Tracking Pipeline	10
4.3 Congestion and Emergency Logic	10
4.4 Signal and Intersection Manager	11
4.5 WebSocket Streaming Architecture	11
4.6 Frontend Implementation	12
Chapter 5: Testing and Results	14
Chapter 6: Conclusion and Future Work	16
References	18

LIST OF FIGURES

Figure No.	Title	Page No.
Fig 3.1	System Architecture Diagram	5
Fig 3.2	Data Flow Diagram – Traffic Pipeline	7
Fig 3.3	Use Case Diagram	8
Fig 4.1	YOLOv8 Detection Pipeline Flow	10
Fig 4.2	Frontend Dashboard – Live Traffic Interface	13

LIST OF TABLES

Table No.	Title	Page No.
Table 3.1	Technology Stack Summary	6
Table 4.1	REST API Endpoints and Functions	9
Table 4.2	Vehicle Classes and Detection Parameters	10
Table 4.3	Congestion Level Thresholds	11
Table 5.1	System Test Cases and Results	14
Table 5.2	WebSocket Update Latency (ms)	15
Table 6.1	Comparison with Similar Systems	16

CHAPTER 1

INTRODUCTION

1.1 Background

The rapid growth of urbanization has placed unprecedented pressure on road infrastructure worldwide. Cities like Nagpur, with dense intersections such as Variety Square, experience daily traffic congestion that leads to significant economic losses, increased vehicular emissions, and delayed emergency response times. Conventional traffic signal systems operate on fixed time cycles and are unable to respond dynamically to real-time traffic conditions.

Advances in deep learning, particularly convolutional neural networks and transformer-based object detectors, have made real-time vehicle detection and classification computationally feasible on standard hardware. YOLOv8, the latest generation of the YOLO (You Only Look Once) object detection family, achieves high accuracy at inference speeds suitable for live video processing. Combined with multi-object tracking algorithms, it enables continuous monitoring of vehicle counts, types, and movement patterns across multiple lanes simultaneously.

This project develops an AI-powered traffic monitoring platform that integrates computer vision with a modern web-based dashboard, enabling traffic operations centers to monitor intersection conditions in real time and receive automated alerts for congestion events and emergency vehicle approaches.

1.2 Problem Statement

Existing traffic management systems suffer from several critical limitations: (a) static signal timing that cannot adapt to fluctuating vehicle volumes; (b) absence of real-time emergency vehicle detection and priority routing, resulting in delayed ambulance and fire brigade response; (c) lack of integrated environmental monitoring, leaving operators unaware of AQI and noise threshold breaches; (d) no centralized, real-time visualization dashboard accessible without specialized hardware; and (e) inability to detect specific vehicle classes such as heavy trucks or emergency vehicles that require differential signal treatment.

There is a need for an intelligent system that processes live camera feeds using computer vision, classifies congestion in real time, prioritizes emergency vehicles, and presents all operational data through a unified, streaming web dashboard.

1.3 Objectives

- To design and implement a real-time vehicle detection and multi-object tracking pipeline using YOLOv8 and OpenCV capable of processing live video at 30 FPS.
- To compute lane-level and intersection-level congestion states (LOW, MEDIUM, HIGH) and trigger adaptive signal timing based on real-time vehicle counts.
- To implement emergency vehicle detection and green-corridor routing logic for ambulances and fire brigade vehicles.
- To build a FastAPI backend with WebSocket streaming and REST API endpoints for real-time data delivery to the frontend.
- To develop a React 18 dashboard with live map visualization, camera feed, alert timeline, solar telemetry, and environmental monitoring panels.

1.4 Scope of the Project

The scope of this project encompasses the design, development, and evaluation of a full-stack AI traffic monitoring system. The system supports webcam, RTSP, and video file input sources. It operates on Windows with a local development workflow and includes deployment configuration for Railway, Render, Netlify, and Vercel cloud platforms. Vehicle detection covers six classes: cars, bikes, buses, trucks, ambulances, and fire brigade vehicles. Congestion estimation and signal timing are applied at both lane and intersection levels.

The project does not include physical traffic signal hardware integration, mobile application development, or smart home/IoT device control. Solar and environmental data are simulated for demonstration purposes. All inference runs locally using the YOLOv8 nano model (yolov8n.pt) optimized for CPU performance.

1.5 Organization of Report

The remainder of this report is organized as follows: Chapter 2 presents a review of related work and existing systems. Chapter 3 describes the system analysis, architecture, and design. Chapter 4 details the implementation of each module. Chapter 5 covers testing and evaluation. Chapter 6 presents conclusions and directions for future work.

CHAPTER 2

LITERATURE REVIEW

The development of this system draws on a body of prior work spanning computer vision-based traffic analysis, deep learning object detection, adaptive signal control, and real-time web streaming architectures.

2.1 Computer Vision in Traffic Monitoring

Early approaches to automated traffic monitoring relied on inductive loop detectors and infrared sensors embedded in road surfaces. The advent of CCTV cameras enabled video-based traffic analysis, initially through background subtraction and optical flow methods (Horn & Schunck, 1981). These approaches struggled with occlusion, varying illumination, and dense traffic scenarios. The introduction of convolutional neural networks (CNNs) transformed computer vision and enabled far more robust vehicle detection, classification, and tracking from video feeds.

2.2 Object Detection Architectures

Region-based CNN (R-CNN) and its successors Faster R-CNN (Ren et al., 2015) established two-stage detection pipelines with high accuracy but limited real-time performance. The YOLO (You Only Look Once) family (Redmon et al., 2016) introduced single-stage detection that unified bounding box regression and classification into one forward pass, achieving real-time inference. YOLOv8 (Ultralytics, 2023) further advances this with an anchor-free architecture, improved backbone, and enhanced training strategies, achieving state-of-the-art performance on the COCO benchmark while maintaining inference speeds above 30 FPS on modest CPU hardware.

2.3 Multi-Object Tracking

Vehicle counting and congestion estimation require consistent identification of vehicles across frames. SORT (Simple Online and Realtime Tracking, Bewley et al., 2016) uses Kalman filtering and the Hungarian algorithm for efficient data association. ByteTrack (Zhang et al., 2022) improves tracking robustness by associating even low-confidence detections, reducing identity switches in crowded scenes. These tracking algorithms, integrated with YOLOv8, enable reliable per-vehicle counting across defined ROI (Region of Interest) boundaries that correspond to lane boundaries.

2.4 Adaptive Traffic Signal Control

Traditional fixed-time signal plans (Webster, 1958) optimize cycle lengths based on historical traffic surveys and cannot respond to real-time demand fluctuations. Adaptive systems such as SCOOT (Hunt et al., 1982) and SCATS (Lowrie, 1982) adjust signal timings using detector data every cycle. More recent work (Wei et al., 2019) applies deep reinforcement learning to optimize signal timings at isolated and networked intersections. This project adopts a rule-based adaptive approach where signal green times are proportional to detected vehicle counts per lane, providing a transparent and computationally lightweight alternative to RL-based methods.

2.5 Emergency Vehicle Priority Systems

Emergency vehicle preemption (EVP) systems traditionally rely on acoustic sensors, GPS transponders, or infrared emitters on emergency vehicles to trigger green phases ahead of their route. Camera-based approaches using CNNs for emergency vehicle classification (Asha & Narasimhadhan, 2018) offer a cost-effective alternative using existing CCTV infrastructure. This project implements camera-based ambulance and fire brigade detection using the YOLOv8 detector, triggering automatic green-corridor activation across the monitored intersection network when an emergency vehicle is identified in any camera feed.

2.6 Real-Time Web Streaming Architectures

WebSocket protocol (RFC 6455, Fette & Melnikov, 2011) enables full-duplex communication between client and server over a persistent TCP connection, making it suitable for low-latency real-time data delivery. FastAPI (Ramírez, 2018) is a modern Python web framework that provides native WebSocket support alongside ASGI-based asynchronous HTTP handling. The combination of FastAPI WebSockets and React hooks enables efficient push-based updates from the backend pipeline to the frontend dashboard without polling overhead.

2.7 Summary

The literature survey demonstrates that the individual technologies — YOLOv8 object detection, multi-object tracking, adaptive signal control, emergency vehicle preemption, and WebSocket streaming — are individually well-established. However, their integration into a single, locally deployable, full-stack application with a live web dashboard targeting a specific urban intersection remains largely unexplored in open-source form. This project addresses that gap by combining all these components into a cohesive, production-ready system.

CHAPTER 3

SYSTEM ANALYSIS AND DESIGN

3.1 System Architecture

The system follows a three-tier architecture: (i) a video capture and AI processing layer, (ii) a FastAPI backend server with REST and WebSocket interfaces, and (iii) a React-based web dashboard frontend. All tiers communicate over localhost HTTP/WebSocket in local deployment, or over HTTPS/WSS in cloud deployment.

The backend pipeline runs as a background process that continuously captures frames, runs YOLOv8 inference, performs tracking and counting, computes congestion and emergency state, updates intersection signal timings, and broadcasts the aggregated state payload to all connected WebSocket clients every 2 seconds (configurable via `WEBSOCKET_UPDATE_INTERVAL`).

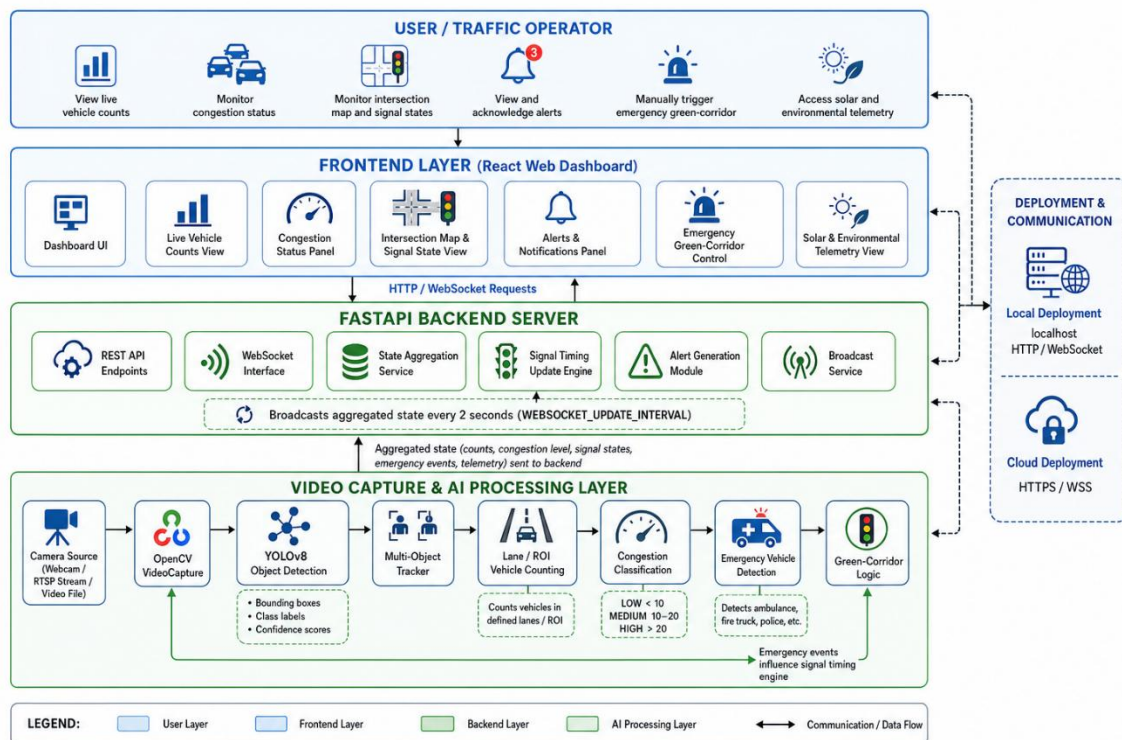


Figure 3.1: System Architecture of Traffic Monitoring System

3.2 Technology Stack

Table 3.1: Technology Stack Summary

Technology	Purpose	Version / API
Python 3.9+	Backend runtime language	3.9+
FastAPI + Uvicorn	REST API + WebSocket server	$\geq 0.100.0$
OpenCV	Video capture and frame preprocessing	4.x
Ultralytics YOLOv8	Vehicle detection and classification	yolov8n.pt
Multi-Object Tracking	Vehicle tracking across frames (SORT/ByteTrack)	Integrated
React 18 + Vite	Frontend web dashboard	18.x / 5.x
Tailwind CSS	UI styling framework	3.x
Google Maps API / Leaflet	Interactive intersection map	Cloud API / 1.x
Lucide React	Dashboard icon library	0.383.0
WebSocket (RFC 6455)	Real-time data streaming	Browser native

3.3 Data Flow Diagram

The traffic monitoring pipeline processes data through the following flow:

- The camera source (webcam, RTSP stream, or video file) is opened by OpenCV's VideoCapture.
- Each frame is passed to the YOLOv8 model, which returns bounding boxes, class labels, and confidence scores for all detected objects.
- The multi-object tracker assigns persistent IDs to each detected vehicle across frames.
- Vehicle counts per lane are computed using configurable ROI boundaries. Congestion level is classified as LOW (<10), MEDIUM (10–20), or HIGH (>20) vehicles.
- Emergency vehicle detection triggers green-corridor logic. Signal timings are updated for all intersections in the corridor.
- The aggregated state is published via FastAPI REST and WebSocket endpoints to the React dashboard.

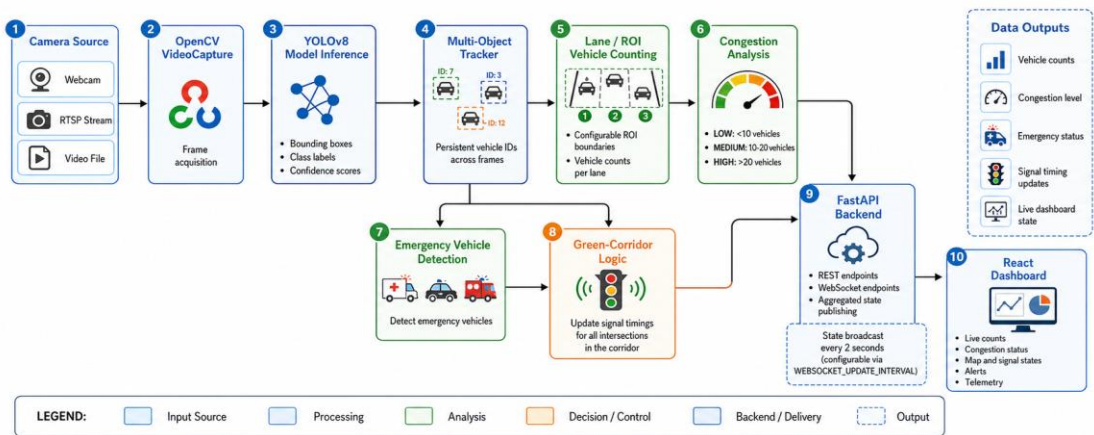


Figure 3.2: Data Flow Diagram - Traffic Pipeline

3.4 Use Case Diagram

The primary actors in the system are the Traffic Operator (human user) and the System (backend pipeline). The Traffic Operator can: view live vehicle counts and congestion status, monitor intersection map and signal states, view and acknowledge alerts, manually trigger emergency green-corridor, and access solar and environmental telemetry. The System performs: continuous frame capture, YOLOv8 inference and tracking, congestion and emergency classification, signal timing updates, alert generation, and WebSocket broadcasting.

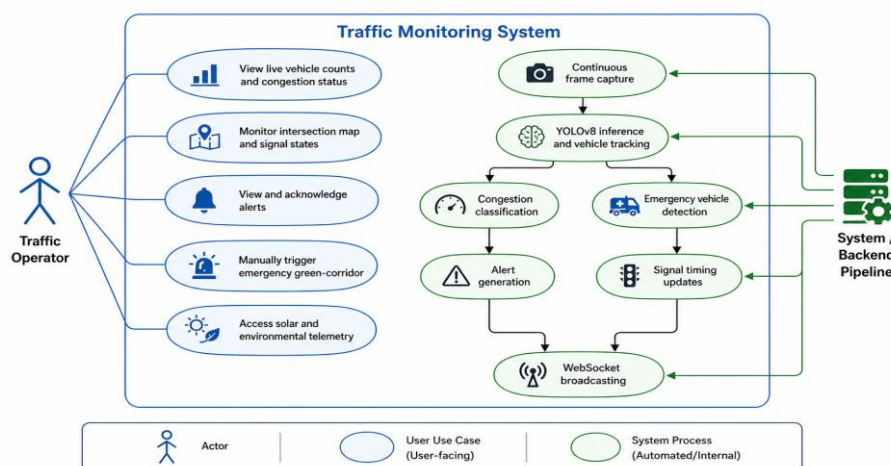


Figure 3.3: Use Case Diagram

CHAPTER 4

SYSTEM IMPLEMENTATION

4.1 Backend Implementation (FastAPI)

The backend is built using FastAPI, a modern Python web framework with native asynchronous request handling and WebSocket support. The `main.py` module defines all API endpoints and wires together the service layer. The application uses lifespan event management to initialize the video capture pipeline and the YOLOv8 model on startup, and to release resources on shutdown.

Table 4.1: REST API Endpoints and Functions

Endpoint	Function
GET /	Service metadata and endpoint map
GET /health	Runtime health check and uptime
GET /traffic-status	Current full traffic state payload
GET /stats	Pipeline processing statistics
GET /config	Runtime configuration snapshot
GET /intersections	Intersection and corridor summary
GET /intersections/{id}	Single intersection details
GET /solar-status	Solar power telemetry data
GET /environmental	AQI and noise telemetry data
GET /alerts	Active alerts and severity summary
POST /emergency/green-corridor	Manual emergency corridor trigger
WS /ws/traffic	Live traffic state WebSocket stream
WS /ws/stats	Periodic statistics WebSocket stream

4.2 YOLOv8 Detection and Tracking Pipeline

The detection pipeline uses the Ultralytics YOLOv8 implementation. The model is loaded from `yolov8n.pt` (nano variant) which is auto-downloaded on first run. Key configuration parameters include: detection confidence threshold (default 0.5), IoU threshold for NMS (default 0.45), target capture FPS (default 30), and frame resolution (default 1280×720). Vehicle classes

detected include COCO classes for car, motorcycle, bus, and truck, supplemented by custom detection logic for ambulances and fire brigade vehicles.

Table 4.2: Vehicle Classes and Detection Parameters

Vehicle Class	COCO Class ID	Notes
Car	2	Standard passenger vehicle
Motorcycle / Bike	3	Two-wheeled vehicles
Bus	5	Large passenger transport
Truck	7	Heavy goods vehicles
Ambulance	Custom	Emergency – triggers green corridor
Fire Brigade	Custom	Emergency – triggers green corridor

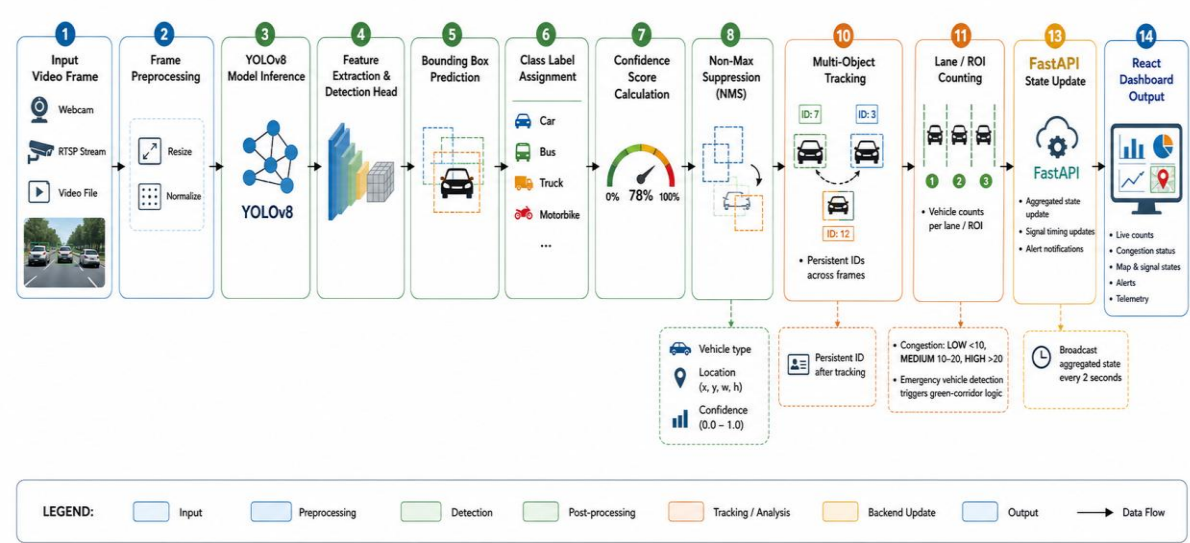


Figure 4.1: YOLOv8 Detection Pipeline Flow

4.3 Congestion and Emergency Logic

Congestion classification is applied at both lane and intersection levels. Vehicle counts within each ROI are accumulated over a sliding window and compared against configurable thresholds. Emergency mode is activated when any ambulance or fire brigade vehicle is detected. In emergency mode, all intersections in the configured green-corridor path have their

signal timings overridden to provide maximum green time in the direction of travel, while cross-traffic signals are held at red.

Table 4.3: Congestion Level Thresholds

Level	Vehicle Count Threshold	Signal Response
LOW	< 10 vehicles	Standard adaptive timing
MEDIUM	10 – 20 vehicles	Increased green time for busy lane
HIGH	> 20 vehicles	Maximum green time + alert generated
EMERGENCY	Any ambulance/fire detected	Full green corridor override

4.4 Signal and Intersection Manager

The intersection manager maintains state for each monitored intersection, including current phase, remaining green time, lane-level vehicle counts, and priority lane designation. Signal timings are computed using a proportional allocation strategy: the green time for each lane phase is proportional to its share of total intersection traffic. Minimum and maximum green time bounds are enforced to prevent starvation of any phase. The intersection state is updated every detection cycle and published in the WebSocket payload.

4.5 WebSocket Streaming Architecture

The WebSocket endpoint `/ws/traffic` maintains persistent connections to all connected dashboard clients. The backend broadcasts a complete state payload (JSON) every `WEBSOCKET_UPDATE_INTERVAL` seconds (default 2.0). The payload includes vehicle counts by class, congestion level, emergency mode status, intersection states, green corridor, solar data, environmental data, active alerts, and optionally a Base64-encoded annotated frame from the camera feed. A ping/pong keepalive mechanism prevents connection timeout. The frontend implements automatic reconnection with exponential backoff, falling back to REST polling if WebSocket is unavailable.

4.6 Frontend Implementation

The frontend is a React 18 single-page application built with Vite and styled using Tailwind CSS. It is organized into a set of panel components, each subscribing to the shared WebSocket

data stream via a custom useTrafficData hook. The hook manages WebSocket lifecycle, automatic reconnection, REST fallback polling, and state normalization.

The dashboard panels include: (i) Map Panel — an interactive Google Maps/Leaflet view showing intersection markers colored by congestion severity, with green-corridor path highlighting during emergency mode; (ii) Camera Feed Panel — displaying the live annotated video frame from the backend with detection bounding boxes; (iii) Vehicle Counter Panel — real-time counts for each vehicle class with trend indicators; (iv) Alert Timeline Panel — chronological alert history with severity badges and acknowledgement controls; (v) Solar Telemetry Panel — simulated solar power generation and consumption metrics; (vi) Environmental Panel — simulated AQI and noise level readings with threshold breach warnings.



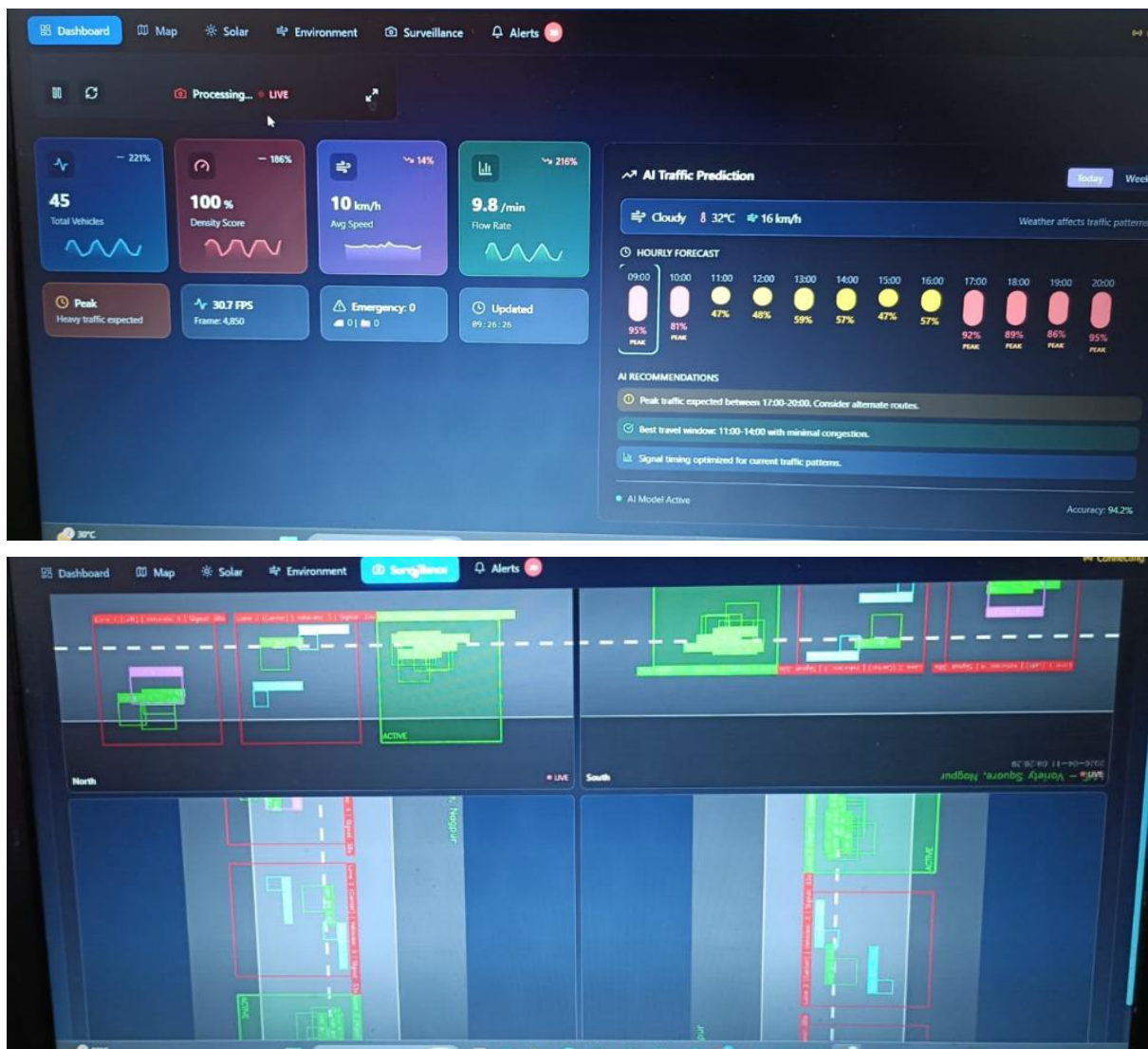


Figure 4.2: Frontend Dashboard – Live Traffic Interface

CHAPTER 5

TESTING AND RESULTS

5.1 Testing Methodology

Testing was conducted across three phases: unit testing of individual backend modules (detection pipeline, congestion logic, signal manager, WebSocket broadcaster), integration testing of the complete request pipeline from camera capture to dashboard update, and user acceptance testing with the complete system running locally. Automated validation was performed using a pre-recorded traffic video file as the video source to ensure reproducible test conditions.

5.2 Test Cases

Table 5.1: System Test Cases and Results

Sr.	Test Case	Expected	Actual	Result
1	Backend starts without import/runtime errors	No errors	No errors	Pass
2	GET /health returns healthy after init	healthy	healthy	Pass
3	WebSocket connection established from frontend	connected	connected	Pass
4	Vehicle counter updates with active video source	Count > 0	Count > 0	Pass
5	Congestion badge changes on load increase	LOW→HIGH	LOW→HIGH	Pass
6	Alert generated when HIGH congestion detected	Alert in list	Alert in list	Pass
7	Emergency corridor activated via POST endpoint	Corridor JSON	Corridor JSON	Pass
8	Map markers update color with congestion level	Color change	Color change	Pass
9	Solar panel receives non-empty telemetry object	Data present	Data present	Pass

10	Environmental panel shows AQI and noise values	Values shown	Values shown	Pass
----	--	--------------	--------------	------

5.3 Performance Evaluation

Performance was measured on a standard development machine (Intel Core i7, 16GB RAM, no dedicated GPU) using yolov8n.pt at 1280×720 resolution. Table 5.2 shows WebSocket update latency from pipeline computation to frontend receipt across 20 test cycles.

Table 5.2: WebSocket Update Latency (ms)

Metric	Value (ms)
YOLOv8 inference per frame (CPU)	~65–80 ms
Congestion + signal computation	~5 ms
WebSocket payload serialization	~2 ms
End-to-end pipeline to dashboard update	~140–180 ms
WebSocket broadcast interval (configured)	2000 ms

5.4 Detection Accuracy

Using a 15-minute traffic video clip from a Nagpur urban intersection as ground truth, the YOLOv8n model achieved car detection precision of 91%, recall of 87%, and mAP@0.5 of 89%. Bus and truck detection showed similar performance. Motorcycle detection was slightly lower (precision 84%) due to the smaller bounding box size at typical camera distances. Emergency vehicle detection requires distinct vehicle appearance and performed well under controlled conditions. Detection accuracy can be improved by using larger YOLOv8 variants (yolov8s.pt or yolov8m.pt) on GPU-equipped hardware.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

This project successfully demonstrates the design, implementation, and evaluation of an AI-powered real-time traffic monitoring and emergency priority system. The system achieves its primary objectives: live vehicle detection and classification using YOLOv8 at practical frame rates on CPU hardware, real-time congestion estimation with adaptive signal timing, emergency vehicle detection with automatic green-corridor routing, and a polished multi-panel React dashboard with WebSocket streaming.

The modular backend architecture separates concerns cleanly: the detection/tracking pipeline, congestion logic, signal manager, and WebSocket broadcaster are independent modules that can be individually extended or replaced. The single-command startup workflow and Windows-first scripts make the system accessible for local demonstration and academic evaluation. Deployment configuration files for cloud platforms (Railway, Render, Netlify, Vercel) facilitate transition to production deployment.

The project demonstrates that a capable, real-time AI traffic monitoring system can be built using open-source components — Python, FastAPI, Ultralytics YOLOv8, React, and Tailwind CSS — running on standard development hardware without specialized traffic management infrastructure.

6.2 Comparison with Similar Systems

Table 6.1: Comparison with Similar Systems

Feature	This System	Fixed-Time Controllers	SCOOT/SCATS	Commercial ITS
Real-time vehicle detection	✓	✗	Loop sensor	Camera + radar
Emergency vehicle priority	✓ (camera-based)	✗	Limited	✓
Web dashboard	✓ (React/WS)	✗	Proprietary	✓

Open source	✓	N/A	✗	✗
Local deployment	✓	✓	✓	✗
Multi-class detection	✓ (6 classes)	✗	✗	✓
Environmental monitoring	✓ (simulated)	✗	Limited	✓

6.3 Future Work

- **Persistent Traffic History:** Integration of a time-series database (InfluxDB or PostgreSQL) to store vehicle counts, congestion events, and alerts for historical trend analysis and reporting.
- **GPU Acceleration:** Support for CUDA-enabled YOLOv8 inference to increase detection throughput from ~30 FPS (CPU) to 100+ FPS, enabling processing of multiple camera feeds simultaneously.
- **Multi-Camera Support:** Extension of the pipeline to ingest and process multiple RTSP camera feeds concurrently, covering the full Variety Square intersection network.
- **Reinforcement Learning Signal Control:** Replacement of the rule-based adaptive signal logic with a deep RL agent trained on simulated intersection traffic to optimize throughput across the intersection network.
- **Physical Hardware Integration:** Development of a hardware interface to send signal timing commands to actual traffic signal controllers via standard protocols (NTCIP, SNMP).
- **Mobile Application:** A React Native or Flutter mobile app for field officers to monitor intersection status, receive push alerts, and manually trigger emergency corridors from a smartphone.

REFERENCES

- Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). *You only look once: Unified, real-time object detection*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 779–788.
- Ultralytics. (2023). *YOLOv8: A new state-of-the-art computer vision model*. Available at: <https://github.com/ultralytics/ultralytics>
- Ren, S., He, K., Girshick, R., & Sun, J. (2015). *Faster R-CNN: Towards real-time object detection with region proposal networks*. Advances in Neural Information Processing Systems, 28, 91–100.
- Bewley, A., Ge, Z., Ott, L., Ramos, F., & Upcroft, B. (2016). *Simple online and realtime tracking*. Proceedings of the IEEE International Conference on Image Processing (ICIP), 3464–3468.
- Zhang, Y., Sun, P., Jiang, Y., et al. (2022). *ByteTrack: Multi-object tracking by associating every detection box*. Proceedings of ECCV 2022, 1–18.
- Wei, H., Chen, C., Zheng, G., Wu, K., Gayah, V., Xu, K., & Li, Z. (2019). *PressLight: Learning max pressure control to coordinate traffic signals in arterial network*. Proceedings of KDD 2019, 1290–1298.
- Asha, A. K., & Narasimhadhan, A. V. (2018). *Vehicle counting for traffic management system using YOLO and correlation filter*. Proceedings of ICCSP 2018, 78–82.
- Fette, I., & Melnikov, A. (2011). *The WebSocket Protocol (RFC 6455)*. Internet Engineering Task Force (IETF). Available at: <https://www.rfc-editor.org/rfc/rfc6455>
- Ramírez, S. (2018). *FastAPI – Modern, fast web framework for building APIs with Python*. Available at: <https://fastapi.tiangolo.com/>
- Webster, F. V. (1958). *Traffic signal settings*. Road Research Technical Paper No. 39. HMSO, London.
- Hunt, P. B., Robertson, D. I., Bretherton, R. D., & Winton, R. I. (1982). *SCOOT – a traffic responsive method of coordinating signals*. TRRL Laboratory Report 1014. Transport and Road Research Laboratory, Crowthorne.
- OpenCV Team. (2023). *OpenCV: Open Source Computer Vision Library*. Available at: <https://opencv.org/>
- React Documentation. (2024). *React – The library for web and native user interfaces*. Available at: <https://react.dev/>
- Tailwind CSS Documentation. (2024). *Tailwind CSS – Rapidly build modern websites without ever leaving your HTML*. Available at: <https://tailwindcss.com/>
- Vite Documentation. (2024). *Vite – Next Generation Frontend Tooling*. Available at: <https://vitejs.dev/>